

Switch Statements in C# with Examples

In this article, I am going to discuss the **Switch Statements in C#** with Examples. Please read our previous articles, where we discussed [If Else Statements in C# Language](#) with Examples. At the end of this article, you will understand what is Switch statement in C# and when and how to use switch statements in C# Language with Examples.

Switch Statements in C# Language:

The switch is a keyword in the C# language, and by using this switch keyword we can create selection statements with multiple blocks. And the Multiple blocks can be constructed by using the case keyword.

Switch case statements in C# are a substitute for long if else statements that compare a variable or expression to several values. The switch statement is a multi-way branching statement which means it provides an easy way to switch the execution to different parts of code based on the value of the expression.

When do we need to go for a switch statement?

When there are several options and we have to choose only one option from the available options depending on a single condition then we need to go for a switch statement. Depending on the selected option a particular task can be performed.

Syntax of Switch Statements in C# Language:

In C#, the Switch statement is a multiway branch statement. It provides an efficient way to transfer the execution to different parts of a code based on the value of the expression. The switch expression is of integer type such as int, byte, or short, or of an enumeration type, or of character type, or of string type. The expression is checked for different cases and the match case will be executed. The following is the syntax to use switch case statement in C# language.

```
switch(variable)
{
    case 1:
        //execute your code
        break;
    case n:
        //execute your code
        break;
    default:
        //execute your code
        break;
}
```

In C#, duplicate case values are not allowed. So, you can create two case statements with the same value. If you try you will get a compilation error.

The default block in the switch statement is optional. That means you can create the switch statements with the default block and, it would run without any problem.

We need to use the break statement inside the switch block to terminate the switch statement execution. That means when the break statement is executed, the switch terminates, and the flow of control jumps to the next line following the switch statement. The break statement is mandatory.

Nesting of switch statements is allowed, which means you can have switch statements inside another switch. However nested switch statements are not recommended by Microsoft. This is because it makes the program more complex and less readable.

Example to understand Switch Statement in C# Language:

```
using System;
namespace ControlFlowDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int x = 2;
            switch (x)
            {
                case 1:
                    Console.WriteLine("Choice is 1");
                    break;
                case 2:
                    Console.WriteLine("Choice is 2");
                    break;
                case 3:
                    Console.WriteLine("Choice is 3");
                    break;
                default:
                    Console.WriteLine("Choice other than 1, 2 and 3");
                    break;
            }
            Console.ReadKey();
        }
    }
}
```

Output: Choice is 2

After the end of each case block, it is necessary to insert a break statement. If we are not inserting the break statement, then we will get a compilation error. But you can combine multiple case blocks with a single break statement if and only if the previous case statement does not have any code block. For a better understanding, please have a look at the below example.

```
using System;
namespace ControlFlowDemo
{
    class Program
    {
```

```

static void Main(string[] args)
{
    string str = "C#";
    switch (str)
    {
        case "C#":
        case "Java":
        case "C":
            Console.WriteLine("It's a Programming Language");
            break;
        case "MSSQL":
        case "MySQL":
        case "Oracle":
            Console.WriteLine("It's a Database");
            break;
        case "MVC":
        case "WEB API":
            Console.WriteLine("It's a Framework");
            break;
        default:
            Console.WriteLine("Invalid Input");
            break;
    }
    Console.ReadKey();
}
}
}

```

Output: **It's a Programming Language**

Is default block Mandatory in a Switch Statement?

No, the default block in the switch statement is not mandatory. If you are putting the default block and if any of the case statement is not fulfilled, then only the default block is going to be executed. For a better understanding, please have a look at the below example where we don't have the default block.

```

using System;
namespace ControlFlowDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string str = "C#2";
            Console.WriteLine("Switch Statement Started");
            switch (str)
            {
                case "C#":
                case "Java":
                case "C":

```

```

        Console.WriteLine("It's a Programming Language");
        break;
    case "MSSQL":
    case "MySQL":
    case "Oracle":
        Console.WriteLine("It's a Database");
        break;
    case "MVC":
    case "WEB API":
        Console.WriteLine("It's a Framework");
        break;
    }
    Console.WriteLine("Switch Statement Ended");
    Console.ReadKey();
}
}
}

```

Output:

Switch Statement Started
Switch Statement Ended

Why do we use Switch Statements instead of if-else statements in C#?

We use the switch statement instead of if-else statements because an if-else statement only works for a small number of logical evaluations of a value. If you use an if-else statement for a larger number of possible conditions then, it takes more time to write and also becomes difficult to understand. For a better understanding, please have a look at the below example. Here, we have written multiple if-else conditions, and, in each condition, we have written the complex expression which not only makes you confused but also it is very difficult to understand.

```

using System;
namespace ControlFlowDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string category;
            // taking topic name
            string topic = "Inheritance";
            if ( topic.Equals("Introduction to C#") ||
                topic.Equals("Variables") ||
                topic.Equals("Data Types"))
            {
                category = "Basic";
            }
            else if (topic.Equals("Loops") ||
                topic.Equals("If ELSE Statements") ||

```

```

        topic.Equals("Jump Statements"))
    {
        category = "Control Flow";
    }
    else if (topic.Equals("Inheritance") ||
        topic.Equals("Polymorphism") ||
        topic.Equals("Abstraction") ||
        topic.Equals("Encapsulation"))
    {
        category = "OOPS Concept";
    }
    else
    {
        category = "Invalid";
    }
    Console.WriteLine($"{topic} Category is {category}");
    Console.ReadKey();
}
}
}

```

Output: Inheritance Category is OOPS Concept

As you can see in the above example, the code is not excessive but, it looks complicated to read and took more time to write. So, instead of using if-else conditions, we can also use a switch statement to save time which is also easier to understand because using a switch statement will provide better readability of code. Let us rewrite the previous example Using Switch Statement in C# language.

```

using System;
namespace ControlFlowDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            string category;
            // taking topic name
            string topic = "Inheritance";
            // using switch Statement
            switch (topic)
            {
                case "Introduction to C#":
                case "Variables":
                case "Data Types":
                    category = "Basic";
                    break;
                case "Loops":
                case "If ELSE Statements":
                case "Jump Statements":
                    category = "Control Flow";
                    break;
            }
        }
    }
}

```

```

        case "Inheritance":
        case "Polymorphism":
        case "Abstraction":
        case "Encapsulation":
            category = "OOPS Concept";
            break;
        // default case
        default:
            category = "Invalid";
            break;
    }
    Console.WriteLine($"{topic} Category is {category}");
    Console.ReadKey();
}
}
}

```

Output: **Inheritance Category is OOPS Concept**

Nested Switch Statement in C#:

Whenever we create a switch statement inside another switch statement, then it is said to be a nested switch statement and this is allowed in C#. Let us see an example to understand this concept.

```

using System;
namespace ControlFlowDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Ask the user to enter a number between 1 and 3
            Console.WriteLine("Enter a Number Between 1 and 3:");
            int number = Convert.ToInt32(Console.ReadLine());
            //outer Switch Statement
            switch (number)
            {
                case 1:
                    Console.WriteLine("You Entered One");
                    //Ask the user to enter the character R, B, or G
                    Console.WriteLine("Enter Color Code (R/G/B): ");
                    char color = Convert.ToChar(Console.ReadLine());
                    //Inner Switch Statement
                    switch (Char.ToUpper(color))
                    {
                        case 'R':
                            Console.WriteLine("You have Selected Red Color");
                            break;
                        case 'G':
                            Console.WriteLine("You have Selected Green Color");

```

```

        break;
    case 'B':
        Console.WriteLine("You have Selected Blue Color");
        break;
    default:
        Console.WriteLine($"You Have Enter Invalid Color (
{Char.ToUpper(color)})");
        break;
    }
    break;
case 2:
    Console.WriteLine("You Entered Two");
    break;
case 3:
    Console.WriteLine("You Entered Three");
    break;
default:
    Console.WriteLine("Invalid Number");
    break;
}
Console.ReadLine();
}
}
}

```

Output:

```

Enter a Number Between 1 and 3:1
You Entered One
Enter Color Code (R/G/B): G
You have Selected Green Color

```

Note: Even though the nested switch statement is allowed, it is not recommended by Microsoft to use nested switch statements. The reason is that the nested switch statements will make your code more complex and less readable.

Although the switch statement makes the code look cleaner than the if...else if statement, the switch is restricted to work with limited data types. The switch statement in C# only works with:

1. **Primitive data types: bool, char, and integral type**
2. **Enumerated Types (Enum)**
3. **String Class**
4. **Nullable types of the above data types**

In the next article, I am going to discuss [Loops in C#](#) with Examples. Here, in this article, I try to explain **Switch Statements in C#** Language with Examples and I hope you like this Switch Statements in C# article. I would like to have your feedback. Please post your feedback, question, or comments about this Control Flow Statements in Java article.